

文本数据处理与序列建模：NLP 数据流程、RNN/LSTM、分类与语言模型

授课教师：May

2025 年 9 月 2 日

目录

1 课程导引与学习目标	3
1.1 课程定位	3
1.2 学习目标	3
2 文本预处理总览 (Normalization & Cleaning)	3
2.1 常见规范化步骤	3
2.2 示例：最小清洗代码 (Python)	3
3 分词与词表 (Tokenization & Vocabulary)	4
3.1 分词粒度选择	4
3.2 子词学习 (BPE) 简化伪代码	4
3.3 特殊符号与建议词表	4
3.4 示例：训练 SentencePiece (子词)	4
4 向量化与嵌入 (Embedding)	5
4.1 表示方式	5
4.2 PyTorch 嵌入示例	5
5 序列对齐与批处理 (Padding, Mask, Packing)	5
5.1 Padding & Truncation 策略	5
5.2 Mask 与忽略 PAD	5
5.3 PackedSequence (RNN/LSTM 专用)	6
6 任务 A：文本分类 (单标签)	6
6.1 问题定义与目标	6
6.2 RNN/LSTM 分类模型 (池化策略)	6
6.3 评测指标	8
6.4 分类任务超参数建议	8

7 任务 B: 语言建模 (下一个 token 预测)	8
7.1 定义与目标	8
7.2 样本构造: 滑动窗口与 TBPTT	8
7.3 LSTM 语言模型 (最小版)	8
7.4 采样与温度/Top-k	10
7.5 语言建模超参数建议	10
8 RNN 与 LSTM: 原理与训练细节	11
8.1 RNN 更新与梯度问题	11
8.2 LSTM 门控机制	11
8.3 LSTM 单元伪代码	11
8.4 实践要点	11
9 NLP 进阶: 中文与多语言注意事项	12
9.1 中文分词与子词	12
9.2 数据划分与泄漏	12
9.3 数据增强 (分类优先)	12
10 数据管线: Dataset/Collate/Loader 模板	12
10.1 统一数据集与对齐函数	12
10.2 语言模型 Dataset (右移标签)	13
11 评测与可视化	13
11.1 分类指标计算	13
11.2 困惑度 (PPL)	14
11.3 Matplotlib 中文字体配置 (可选)	14
12 常见错误与排查清单	14
13 附录: 公式与实现补充	14
13.1 RNN 梯度消失直观推导	14
13.2 带注意力掩码的序列交叉熵 (伪代码)	15
13.3 早停与学习率调度 (示意)	15
13.4 LSTM 与 GRU 对比 (要点)	15

1 课程导引与学习目标

1.1 课程定位

本讲义系统梳理 NLP 文本训练数据集的处理流程，并将流程与 RNN/LSTM 模型训练在两类典型任务——文本分类与语言建模（下一个 token 预测）——中一一对应，兼顾数学公式、工程实现与调参与诊断。

1.2 学习目标

- 掌握从原始文本到可训练张量的完整管线（清洗、分词、词表、向量化、对齐、批处理、掩码）。
- 了解 RNN/LSTM 的更新方程、训练技巧（梯度裁剪、TBPTT、PackedSequence）。
- 会为 分类与语言建模 构建数据集、损失函数与评测指标（F1/困惑度）。
- 会在中文/多语言场景中做分词与规范化（全/半角、Unicode、子词）。

2 文本预处理总览 (Normalization & Cleaning)

2.1 常见规范化步骤

1. 空白与大小写：统一空白（多空格 → 单空格），英文统一小写（如不区分专有名词）。
2. Unicode 规范化：使用 NFKC/NFKD 归一化，处理变体字符（如全角/半角）。
3. 标点与符号：统一中英文标点，保留与任务相关的标点（如情感分类保留“！”）。
4. 去噪：去除 HTML/URL/用户提及/表情（或将其替换为占位符 <URL>）。
5. 语言特定：中文可不做词形变化；英文可做词干化/词形还原、去停用词（任务相关）。

2.2 示例：最小清洗代码 (Python)

```
1 import re
2 import unicodedata
3
4 def normalize_text(s: str) -> str:
5     s = unicodedata.normalize("NFKC", s)           # Unicode 规范化
6     s = re.sub(r"https?://\S+|www\.\S+", "<URL>", s)
7     s = re.sub(r"@w+", "<USER>", s)                # 社交媒体 @提及
8     s = re.sub(r"\s+", " ", s).strip()             # 压缩空白
9     return s
```

Listing 1: 基础正则清洗示例

教学提示：清洗强度与任务挂钩——垃圾短信检测可更激进，法律/医疗任务需保留更多原文细节。

3 分词与词表 (Tokenization & Vocabulary)

3.1 分词粒度选择

- **字符级 (char)**: OOV 为零，序列更长；适合小词表基线或极多语言混杂。
- **词级 (word)**: 直观但 OOV 严重；中文需外部分词器 (jieba/THULAC)。
- **子词 (subword)**: BPE/WordPiece/UnigramLM，平衡 OOV 与序列长度，是现代主流。

3.2 子词学习 (BPE) 简化伪代码

Algorithm 1 BPE 学习 (简化)

- 1: 将语料切成字符序列 (词尾加 `_` 表示边界)
 - 2: **for** 迭代直到词表上限 V **do**
 - 3: 统计所有相邻符号对 (a, b) 的频次
 - 4: 合并最高频对 $(a^*, b^*) \rightarrow \langle ab \rangle$
 - 5: 用新符号替换语料中的 (a^*, b^*)
 - 6: **end for**
 - 7: 输出合并规则与最终子词词表
-

3.3 特殊符号与建议词表

- 特殊符号: `<PAD>`, `<UNK>`, `<BOS>`, `<EOS>`, `<CLS>`, `<SEP>`, `<MASK>`
- 词表规模 (经验): 中文子词 16k ~ 32k; 英文子词 30k ~ 50k

3.4 示例: 训练 SentencePiece (子词)

```
1 # pip install sentencepiece
2 import sentencepiece as spm
3
4 # 训练: --model_type 可选 bpe/unigram
5 spm.SentencePieceTrainer.Train(
6     input="corpus.txt",
7     model_prefix="spm_bpe",
8     vocab_size=32000,
9     character_coverage=0.9995, # 中文建议 0.9995 以上
```

```

10 )
11
12 # 使用
13 sp = spm.SentencePieceProcessor(model_file="spm_bpe.model")
14 ids = sp.encode("深度学习很有趣。", out_type=int)
15 text = sp.decode(ids)

```

Listing 2: SentencePiece 训练与使用最小示例

4 向量化与嵌入 (Embedding)

4.1 表示方式

- **One-hot**: 高维稀疏, 不含语义关系。
- **分布式词向量**: Word2Vec/GloVe/FastText, 或随机初始化的 `nn.Embedding`。
- **上下文表示**: 由 RNN/LSTM/Transformer 产生的动态表征。

4.2 PyTorch 嵌入示例

```

1 import torch, torch.nn as nn
2
3 vocab_size, embed_dim, pad_id = 32000, 300, 0
4 emb = nn.Embedding(vocab_size, embed_dim, padding_idx=pad_id)
5 x = torch.tensor([[5, 7, 0, 0], [9, 11, 13, 0]]) # 0 为 <PAD>
6 e = emb(x) # [batch, seq, embed_dim]

```

Listing 3: `nn.Embedding` 与 `padding_idx`

5 序列对齐与批处理 (Padding, Mask, Packing)

5.1 Padding & Truncation 策略

- **头截断/尾截断**: 保留文本前部或后部 (情感倾向常保留后部)。
- **长度分桶 (bucketing)**: 按长度分组, 减少填充, 提升吞吐。

5.2 Mask 与忽略 PAD

```

1 import torch
2 from torch.nn.utils.rnn import pad_sequence
3

```

```

4 PAD = 0
5 batch = [torch.tensor([3,4,5]), torch.tensor([6,7])]
6 pad = pad_sequence(batch, batch_first=True, padding_value=PAD)    #
    [[3,4,5],[6,7,0]]
7 attn_mask = (pad != PAD).int()  # 1=有效, 0=pad
8
9 # 语言建模损失时:
10 ignore_index = PAD
11 loss_fn = torch.nn.CrossEntropyLoss(ignore_index=ignore_index)

```

Listing 4: 构造 Attention Mask 与忽略 PAD 损失

5.3 PackedSequence (RNN/LSTM 专用)

```

1 from torch.nn.utils.rnn import pack_padded_sequence,
    pad_packed_sequence
2
3 lengths = torch.tensor([3,2], dtype=torch.long) # 按降序排序
4 packed = pack_padded_sequence(pad, lengths, batch_first=True,
    enforce_sorted=True)
5 out_packed, _ = lstm(packed)
6 out, out_lens = pad_packed_sequence(out_packed, batch_first=True)

```

Listing 5: pack_padded_sequence 使用示例

6 任务 A: 文本分类 (单标签)

6.1 问题定义与目标

给定 token 序列 $x_{1:T}$, 预测类别 $y \in \{1, \dots, C\}$:

$$y^* = \arg \max_y P(y \mid x_{1:T}), \quad \mathcal{L}_{\text{CE}} = - \sum_{c=1}^C \mathbf{1}(y=c) \log \hat{p}_c$$

6.2 RNN/LSTM 分类模型 (池化策略)

常见输出聚合: 最后状态、平均池化、最大池化、[CLS] 位。

```

1 import torch, torch.nn as nn, torch.optim as optim
2
3 class LSTMClassifier(nn.Module):
4     def __init__(self, vocab, embed_dim=300, hidden=256, layers=1,
        num_classes=2, pad_id=0, bidir=True, drop=0.3):

```

```

5      super().__init__()
6      self.emb = nn.Embedding(vocab, embed_dim, padding_idx=pad_id)
7      self.lstm = nn.LSTM(embed_dim, hidden, num_layers=layers,
8                          batch_first=True,
9                          dropout=(drop if layers>1 else 0.0),
10                         bidirectional=bidir)
11      out_dim = hidden * (2 if bidir else 1)
12      self.dropout = nn.Dropout(drop)
13      self.fc = nn.Linear(out_dim, num_classes)
14
15  def forward(self, x, lengths):
16      x = self.emb(x)
17      packed = nn.utils.rnn.pack_padded_sequence(x, lengths.cpu(),
18          batch_first=True, enforce_sorted=True)
19      out, (h, c) = self.lstm(packed)
20      # 使用最后层双向的最后隐藏状态拼接
21      if self.lstm.bidirectional:
22          feat = torch.cat([h[-2], h[-1]], dim=-1)
23      else:
24          feat = h[-1]
25      feat = self.dropout(feat)
26      return self.fc(feat)
27
28 # 训练循环 (含梯度裁剪与类别不均衡权重)
29 model = LSTMClassifier(vocab=32000, num_classes=3)
30 criterion = nn.CrossEntropyLoss(weight=torch.tensor([1.0,1.5,2.0])) #
31     示例权重
32 optimizer = optim.Adam(model.parameters(), lr=2e-3)
33
34 for epoch in range(5):
35     model.train()
36     for pad_ids, lengths, labels in train_loader:
37         optimizer.zero_grad()
38         logits = model(pad_ids, lengths)
39         loss = criterion(logits, labels)
40         loss.backward()
41         nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
42         optimizer.step()

```

Listing 6: LSTM 文本分类最小实现

6.3 评测指标

- 准确率、精确率/召回率/F1（宏/微/加权）。
- 分层样本不平衡时优先 **宏平均 F1**。

6.4 分类任务超参数建议

超参数	推荐范围	说明
词向量维度	100–300	语料大可取更高
隐藏维度（LSTM）	128–512	与类别数/数据量相关
层数	1–3	层数越多越需正则化
Dropout	0.2–0.5	防过拟合
学习率	$1e-4 \sim 3e-3$	Adam/AdamW 常用
Batch size	32–128	结合显存与长度分桶
序列最大长度	128/256/512	任务与语料统计决定
梯度裁剪	1.0–5.0	抑制梯度爆炸
类不平衡	加权 CE/焦点损失	或重采样/阈值移动

表 1: 文本分类超参数指南

7 任务 B：语言建模（下一个 token 预测）

7.1 定义与目标

$$\max_{\theta} \sum_{t=1}^T \log P_{\theta}(x_t | x_{<t}), \quad \text{PPL} = \exp\left(\frac{1}{N} \sum_{i=1}^N \mathcal{L}_i\right)$$

其中 PPL（困惑度）越低越好。

7.2 样本构造：滑动窗口与 TBPTT

- 输入： $[x_1, \dots, x_n]$ ；目标： $[x_2, \dots, x_{n+1}]$ （右移一位）。
- TBPTT：按固定长度截断反传，控制显存与依赖跨度。

7.3 LSTM 语言模型（最小版）

```
1 import torch, torch.nn as nn, torch.optim as optim
2
3 class LSTMLM(nn.Module):
```



```

4     def __init__(self, vocab, emb=300, hidden=512, layers=2, pad_id=0,
5         drop=0.3):
6         super().__init__()
7         self.emb = nn.Embedding(vocab, emb, padding_idx=pad_id)
8         self.lstm = nn.LSTM(emb, hidden, num_layers=layers, batch_first
9             =True, dropout=(drop if layers>1 else 0.))
10        self.drop = nn.Dropout(drop)
11        self.out = nn.Linear(hidden, vocab)
12
13    def forward(self, x, h=None):
14        e = self.emb(x)
15        y, h = self.lstm(e, h)
16        y = self.drop(y)
17        logits = self.out(y) # [B, T, V]
18        return logits, h
19
20    def tbptt_iter(data, bptt=128):
21        # data: [num_batches, batch, seq_len]
22        for batch in data:
23            for i in range(0, batch.size(1) - 1, bptt):
24                x = batch[:, i:i+bptt]
25                y = batch[:, i+1:i+bptt+1]
26                yield x, y
27
28    vocab, PAD = 32000, 0
29    model = LSTMLM(vocab)
30    crit = nn.CrossEntropyLoss(ignore_index=PAD)
31    opt = optim.AdamW(model.parameters(), lr=2e-3)
32
33    for epoch in range(5):
34        h = None
35        for x, y in tbptt_iter(train_batches, bptt=128):
36            opt.zero_grad()
37            logits, h = model(x, h)
38            # 截断梯度历史, 避免跨段反向图堆积
39            h = tuple(s.detach() for s in h)
40            loss = crit(logits.reshape(-1, vocab), y.reshape(-1))
41            loss.backward()
42            nn.utils.clip_grad_norm_(model.parameters(), 1.0)
43            opt.step()

```

Listing 7: LSTM 语言模型与 TBPTT

7.4 采样与温度/Top-k

```
1 import torch
2 import torch.nn.functional as F
3
4 @torch.no_grad()
5 def sample(model, start, max_len=100, temperature=1.0, top_k=50):
6     ids = start.clone()
7     h = None
8     for _ in range(max_len):
9         logits, h = model(ids[:, -1:].contiguous(), h)
10        logits = logits[:, -1, :] / max(temperature, 1e-6)
11        # top-k 过滤
12        topv, topi = torch.topk(logits, k=top_k, dim=-1)
13        probs = torch.zeros_like(logits).scatter_(-1, topi, F.softmax(
14            topv, dim=-1))
15        next_id = torch.multinomial(probs, num_samples=1)
16        ids = torch.cat([ids, next_id], dim=1)
17    return ids
```

Listing 8: 从 LSTM LM 采样文本（温度与 top-k）

7.5 语言建模超参数建议

超参数	推荐范围	说明
嵌入维度	256–768	语料越大取值越高
隐藏维度 (LSTM)	512–1024	生成质量与容量相关
层数	2–4	更深需更强正则
BPTT 长度	64–256	依设备显存与依赖跨度
学习率	$1e-4 \sim 3e-3$	AdamW + 线性 warmup
Dropout	0.2–0.6	嵌入/层间/输出均可用
梯度裁剪	0.5–1.0	抑制爆炸
Batch size	16–128	取决于序列长度
采样策略	温度 0.7–1.2	配合 top-k/top-p

表 2: 语言建模超参数指南

8 RNN 与 LSTM: 原理与训练细节

8.1 RNN 更新与梯度问题

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b), \quad \frac{\partial L}{\partial W_h} \propto \prod_{k=1}^t \frac{\partial h_k}{\partial h_{k-1}}$$

若 $\|\frac{\partial h_k}{\partial h_{k-1}}\| < 1$ 则梯度消失, > 1 则爆炸。

工程对策 梯度裁剪、正交初始化、残差/层归一化、短截反传 (TBPTT)、门控结构 (LSTM/GRU)。

8.2 LSTM 门控机制

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad (1)$$

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c), \quad C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \quad (2)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(C_t). \quad (3)$$

8.3 LSTM 单元伪代码

Algorithm 2 LSTM 单元 (前向)

```
1: Input:  $x_t, h_{t-1}, C_{t-1}$ 
2:  $f_t \leftarrow \sigma(W_f[h_{t-1}, x_t] + b_f)$ 
3:  $i_t \leftarrow \sigma(W_i[h_{t-1}, x_t] + b_i)$ 
4:  $\tilde{C}_t \leftarrow \tanh(W_c[h_{t-1}, x_t] + b_c)$ 
5:  $C_t \leftarrow f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$ 
6:  $o_t \leftarrow \sigma(W_o[h_{t-1}, x_t] + b_o)$ 
7:  $h_t \leftarrow o_t \odot \tanh(C_t)$ 
8: Return:  $h_t, C_t$ 
```

8.4 实践要点

- **双向 RNN/LSTM:** 分类常用以捕获双向上下文; 语言建模通常使用单向 (因因果约束)。
- **Dropout:** PyTorch RNN/LSTM 的 dropout 作用于层间输出 (层数 > 1 时生效)。
- **LayerNorm/LN-LSTM:** 稳定训练, 减小梯度方差。

9 NLP 进阶：中文与多语言注意事项

9.1 中文分词与子词

- 中文无空格天然分词单位，建议采用 子词 (SentencePiece BPE/Unigram) 统一中英符号。
- 需处理全/半角、中文标点、数字与单位 (例如 “1万”，“\$”)。

9.2 数据划分与泄漏

- 按文档/用户/时间切分，避免同主体出现在训练与验证。
- 文本去重与近重复检测 (SimHash/MinHash) 减少数据泄漏。

9.3 数据增强 (分类优先)

- EDA: 同义替换、随机插入/删除/交换 (中文可基于同义词词林/词向量近邻)。
- 回译 (zh→en→zh): 较稳健但成本较高。
- Token Dropout / 随机掩码: 以小概率替换为 <MASK> 或 <UNK>。

10 数据管线：Dataset/Collate/Loader 模板

10.1 统一数据集与对齐函数

```
1 from torch.utils.data import Dataset, DataLoader
2 from torch.nn.utils.rnn import pad_sequence
3 import torch
4
5 PAD = 0
6
7 class ClsDataset(Dataset):
8     def __init__(self, texts, labels, sp):
9         self.texts, self.labels, self.sp = texts, labels, sp
10    def __len__(self): return len(self.texts)
11    def __getitem__(self, i):
12        ids = self.sp.encode(self.texts[i], out_type=int)
13        return torch.tensor(ids), torch.tensor(self.labels[i])
14
15 def collate_batch(samples):
16     seqs, labels = zip(*samples)
17     lengths = torch.tensor([len(s) for s in seqs], dtype=torch.long)
```

```

18     # 按长度降序 (便于 pack)
19     order = torch.argsort(lengths, descending=True)
20     seqs = [seqs[i] for i in order]; labels = torch.stack([labels[i]
21         for i in order])
22     lengths = lengths[order]
23     pad = pad_sequence(seqs, batch_first=True, padding_value=PAD)
24     return pad, lengths, labels
25
26 loader = DataLoader(ClsDataset(train_texts, train_labels, sp),
27     batch_size=64,
28     shuffle=True, collate_fn=collate_batch)

```

Listing 9: 文本分类 Dataset 与 collate_fn

10.2 语言模型 Dataset (右移标签)

```

1 class LMDataset(Dataset):
2     def __init__(self, text, sp, seq_len=128):
3         ids = sp.encode(text, out_type=int)
4         self.x, self.y = [], []
5         for i in range(0, len(ids)-seq_len-1, seq_len):
6             seq = ids[i:i+seq_len]
7             tgt = ids[i+1:i+seq_len+1]
8             self.x.append(torch.tensor(seq)); self.y.append(torch.
9                 tensor(tgt))
10    def __len__(self): return len(self.x)
11    def __getitem__(self, i): return self.x[i], self.y[i]

```

Listing 10: LM 样本切分 (定长窗口)

11 评测与可视化

11.1 分类指标计算

```

1 import torch
2
3 def macro_f1(tp, fp, fn, n_classes):
4     f1s = []
5     for c in range(n_classes):
6         p = tp[c] / max(tp[c]+fp[c], 1)
7         r = tp[c] / max(tp[c]+fn[c], 1)
8         f1 = 2*p*r / max(p+r, 1e-12)

```

```

9         f1s.append(f1)
10     return sum(f1s)/len(f1s)

```

Listing 11: 精确率/召回率/F1 (宏)

11.2 困惑度 (PPL)

$$\text{PPL} = \exp\left(\frac{1}{N} \sum_{i=1}^N \mathcal{L}_i\right), \quad \mathcal{L}_i = \text{CE}(\hat{p}(x_t | x_{<t}), x_t)$$

11.3 Matplotlib 中文字体配置 (可选)

```

1 import matplotlib.pyplot as plt
2 plt.rcParams['font.sans-serif'] = ['SimHei', 'Noto Sans CJK SC']
3 plt.rcParams['axes.unicode_minus'] = False

```

Listing 12: 中文字体配置

12 常见错误与排查清单

- **PAD 未忽略**: 交叉熵未设置 `ignore_index` 导致损失异常。
- **长度未排序**: 使用 `pack_padded_sequence` 时未降序排序。
- **泄漏**: 同一用户/文档切分到训/验/测。
- **过度清洗**: 删除关键标点或表情, 损伤情感信号。
- **类不平衡**: 仅看准确率, 忽视 F1/ROC-AUC。
- **梯度爆炸**: RNN/LSTM 未裁剪梯度, 训练不稳定。

13 附录: 公式与实现补充

13.1 RNN 梯度消失直观推导

设 $h_t = \phi(W h_{t-1} + U x_t)$, 则

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{j=t-k+1}^t \phi'(a_j) W$$

若 $\|\phi'(a_j)W\| < 1$, 连乘趋近 0。

13.2 带注意力掩码的序列交叉熵（伪代码）

```
1 # logits: [B,T,V], target: [B,T], pad=0
2 loss = CE(logits.view(-1, V), target.view(-1), ignore_index=pad)
```

Listing 13: 掩码 CE（忽略 PAD）

13.3 早停与学习率调度（示意）

```
1 best = 1e9; bad_epochs = 0
2 for epoch in range(50):
3     train_one_epoch(...)
4     val_loss = evaluate(...)
5     if val_loss < best - 1e-4:
6         best, bad_epochs = val_loss, 0
7         save_checkpoint()
8     else:
9         bad_epochs += 1
10        scheduler.step(val_loss) # ReduceLROnPlateau
11    if bad_epochs >= 5: break
```

Listing 14: EarlyStopping + ReduceLROnPlateau

13.4 LSTM 与 GRU 对比（要点）

- GRU 结构更简洁，参数更少，训练更快；长依赖上 LSTM 更稳健（经验视任务）。
- 文本分类常用 BiLSTM/BiGRU；语言建模常用单向 LSTM/GRU。